

Arquitetura de Microsserviços e Mapa de Domínios

Santo Pegasus Soluciones

Documento: Arquitetura de Microsserviços e Mapa de Domínios\

Versão: 1.0.0\

Data de Emissão: Junho de 2026\

Departamento: Engenharia de Software / Chapter de Backend\

Classificação: Interno — Uso Técnico Restrito

TABLA DE CONTEÚDOS

- 1. Introdução e Visão Geral da Arquitetura
- 2. Diagrama Textual da Arquitetura Geral
- 3. Catálogo Completo de Microsserviços
- 4. Mapa de Dependências entre Serviços
- 5. Padrões de Comunicação
- 6. Estratégia de Bancos de Dados
- 7. API Gateway
- 8. Infraestrutura na AWS
- 9. Observabilidade Distribuída
- 10. Estratégia de Versionamento de APIs
- 11. Segurança entre Serviços
- 12. Mapa de Squads e Ownership
- 13. Roadmap Técnico
- 14. Architecture Decision Records (ADRs)
- 15. Disposições Finais e Processo de Atualização

SEÇÃO 1 — INTRODUÇÃO E VISÃO GERAL DA ARQUITETURA

1.1 Propósito do Documento

Este documento constitui o mapa arquitetônico oficial da Santo Pegasus Soluciones. Seu propósito é servir como referência canônica para todos os engenheiros, Tech Leads, Product Managers e stakeholders técnicos que precisem compreender como estão organizados, interconectados e implantados os sistemas que compõem o portfólio de produtos da empresa. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

Este artefato deve ser lido em conjunto com o Guia Oficial de Engenharia Backend, que estabelece os padrões de codificação, padrões de design e práticas de segurança obrigatórias para todos os serviços aqui descritos. Ambos os documentos formam o núcleo da base de conhecimento técnico da Santo Pegasus e são fontes de verdade para a tomada de decisões arquitetônicas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

1.2 Contexto Empresarial

A Santo Pegasus Soluciones é uma empresa de tecnologia especializada no desenvolvimento de produtos digitais para o setor de saúde e serviços profissionais. O produto principal da companhia é o Agendio, uma plataforma SaaS de agendamento de consultas médicas que conecta pacientes, médicos e clínicas por meio de uma experiência digital integrada. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

A plataforma Agendio opera em modelo multi-tenant, atendendo a redes de clínicas, hospitais de pequeno e médio porte, e consultórios independentes em todo o Brasil. A criticidade do domínio de saúde exige que a arquitetura priorize disponibilidade, segurança de dados sensíveis, conformidade com a LGPD (Lei Geral de Proteção de Dados) e auditabilidade completa de todas as operações. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

1.3 Por Que Microserviços

A adoção de uma arquitetura de microserviços na Santo Pegasus não foi uma decisão tomada de forma prematura. A empresa começou com um monolito funcional, e a migração para microserviços foi guiada por necessidades reais e imediatas de escala, autonomia de equipes e velocidade de entrega, em consonância com os princípios fundamentais documentados no Guia de Engenharia Backend. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

As motivações concretas que justificaram a transição são:

- **Escalabilidade Independente:** O serviço de agendamento (`agendio-scheduling-service`) apresenta picos de carga nos horários matutinos (entre 07h00 e 09h00) que não impactam outros domínios. Em arquitetura monolítica, escalar esse módulo implicaria escalar toda a aplicação, desperdiçando recursos computacionais e aumentando custos operativos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]
- **Autonomia de Squads:** Com equipes organizadas por domínio de negócio, a arquitetura de microserviços permite que cada squad tenha ownership completo de seu ciclo de vida de software — desde o código até o deploy em produção — sem coordenação burocrática com outras equipes para releases. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 2\]

- **Resiliência e Isolamento de Falhas:** Um defeito crítico no `payment-service` não deve causar a indisponibilidade do agendamento de consultas. O isolamento de processos garante que as falhas sejam contidas dentro do domínio afetado. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

- **Flexibilidade Tecnológica:** Diferentes domínios têm diferentes requisitos técnicos. O `medical-records-service` se beneficia do MongoDB para o armazenamento de documentos clínicos flexíveis e semiestruturados, enquanto o `auth-service` requer a consistência forte do PostgreSQL para a gestão de credenciais. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

- **Velocidade de Entrega:** A independência de deploys permite que múltiplas squads liberem novas funcionalidades no mesmo dia sem coordenação de releases nem janelas de manutenção compartilhadas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

1.4 Princípios Arquitetônicos que Guiam as Decisões

Todas as decisões arquitetônicas na Santo Pegasus estão ancoradas nos seguintes princípios: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 3\]

#	Princípio	Descrição
P-01	High Cohesion, Low Coupling	Cada microserviço deve encapsular um domínio de negócio bem definido. Dependências entre serviços devem ser minimizadas e, quando existentes, preferencialmente assíncronas.
P-02	Database per Service	Cada microserviço possui e controla exclusivamente seu próprio banco de dados. O acesso direto ao banco de dados de outro serviço é terminantemente proibido.
P-03	API First	Contratos de API (OpenAPI/Swagger) são definidos antes da implementação. Isso garante que os consumidores possam desenvolver em paralelo usando mocks.
P-04	Security by Default	Todas as comunicações entre serviços são autenticadas. Nenhum endpoint interno é acessível sem validação de identidade. mTLS é obrigatório para comunicação interna.

P-05	Observability by Design	Logs estruturados, métricas e rastreamento distribuído não são adicionados post-hoc; são parte do scaffolding inicial de cada novo serviço.
P-06	Fail Fast, Recover Gracefully	Os serviços implementam circuit breakers, timeouts e políticas de retry com backoff exponencial para garantir resiliência diante de falhas de dependências.
P-07	Compliance First	Decisões sobre armazenamento, transmissão e processamento de dados sensíveis de saúde são guiadas pela LGPD e boas práticas de segurança antes de qualquer requisito funcional.
P-08	Infrastructure as Code	Toda a infraestrutura AWS é definida como código (Terraform/AWS CDK). Nenhum recurso de produção é criado manualmente via Console.
P-09	Evolutionary Architecture	A arquitetura aceita que mudará. Decisões são tomadas com horizontes de tempo claros e revisadas periodicamente nos fóruns de Architecture Review.
P-10	SOLID in Every Service	Os princípios SOLID de orientação a objetos são aplicados tanto a nível de classe quanto a nível de serviço. Cada microserviço tem uma única responsabilidade de domínio.

1.5 Ecossistema Tecnológico Principal

O ecossistema tecnológico da Santo Pegasus é padronizado para garantir consistência operacional e reduzir a carga cognitiva das equipes: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 4\]

- **Linguagem & Runtime: Java 17+ (LTS), com adoção gradual do Java 21 (Virtual Threads).**
- **Framework Principal: Spring Boot 3+, Spring Security, Spring Cloud.**
- **Containerização: Docker (imagens imutáveis), orquestrados no AWS ECS Fargate.**
- **Bancos de Dados: PostgreSQL (relacional), MongoDB/AWS DocumentDB (documentos), Redis/AWS ElastiCache (cache).**
- **Mensageria: AWS SQS (filas e eventos entre domínios).**
- **Autenticação: JWT + OAuth 2.0 / OpenID Connect, gerenciado pelo `auth-service`.**
- **Observabilidade: SLF4J + Logback, Spring Boot Actuator, Micrometer, Prometheus, Datadog.**
- **CI/CD: GitHub Actions + Pipelines com Docker, Flyway/Liquibase para migrations.**

- **Secrets Management: AWS Secrets Manager + Spring Cloud Config.**

SEÇÃO 2 — DIAGRAMA TEXTUAL DA ARQUITETURA GERAL

2.1 Visão de Alto Nível

O diagrama a seguir representa a topologia completa da arquitetura de microsserviços da Santo Pegasus: \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 5\]

...

CLIENTES EXTERNOS

[Web App React] [Mobile App iOS/Android] [Parceiros de API de Clínicas]

▼ HTTPS / TLS 1.3

AWS API GATEWAY (Kong / AWS API GW)

- Routing • Rate Limiting • Auth Token Validation
- SSL Termination • CORS • Request Logging

▼▼▼▼▼

auth- user- agendio- payment- ai-assistant-

service service scheduling service service

:8081 :8082 -service :8085 :8087

:8083

▼▼▼▼▼

[PostgreSQL] [PostgreSQL] [PostgreSQL] [Pinecone DB]

auth_db users_db payments_db (Vector Store)

▼▼▼

agendio- medical- audit-

notif- records- service

service service :8088

:8084 :8086

▼

▼▼ [PostgreSQL]

[AWS SES] [MongoDB / audit_db

[AWS SNS] DocumentDB]

medical_records_db

CAMADA DE MENSAGERIA ASSÍNCRONA (AWS SQS QUEUES)

appointment-created.fifo → [agendio-notification-service]

appointment-cancelled.fifo → [agendio-notification-service] & [audit-service]

payment-confirmed.fifo → [agendio-scheduling-service] & [audit-service]

user-created.fifo → [agendio-notification-service] & [audit-service]

audit-events.fifo → [audit-service]

INFRAESTRUTURA COMPARTILHADA

AWS ElastiCache (Redis) | AWS CloudWatch + Datadog (Observability) | AWS Secrets Manager + Spring Cloud Config

...

2.2 Fluxo de uma Requisição Típica (Agendamento de Consulta)

1. Cliente (App Mobile): `POST /v1/appointments [Bearer JWT]`
2. API Gateway: Valida JWT via `auth-service` (cache Redis), aplica rate limit e roteia para o `agendio-scheduling-service`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
3. agendio-scheduling-service: Valida disponibilidade do médico, verifica perfil do paciente (REST síncrono), cria registro no `scheduling\_db`, publica evento "appointment-created" no SQS e retorna `201 Created`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
4. \[Async\] agendio-notification-service: Consome SQS e envia e-mail/SMS de confirmação via AWS SES/SNS. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]
5. \[Async\] audit-service: Consome SQS e registra a ação no `audit\_db` com Trace ID completo. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]

SEÇÃO 3 — CATÁLOGO COMPLETO DE MICROSERVIÇOS

3.1 Resumo do Catálogo \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 7\]

Serviço	Porta	Domínio	Banco de Dados	Squad Owner
auth-service	8081	Identidade & Segurança	PostgreSQL	Squad Hermes
user-service	8082	Usuários & Perfis	PostgreSQL	Squad Hermes
agendio-scheduling-service	8083	Agendamento (Core)	PostgreSQL	Squad Agendio Core
agendio-notificatio n-service	8084	Notificações	— (stateless)	Squad Agendio Core
payment-service	8085	Pagos & Faturamento	PostgreSQL	Squad Pagamentos

medical-records-service	8086	Histórico Clínico	MongoDB	Squad Clínico
ai-assistant-service	8087	Assistência IA (RAG)	Pinecone (Vector)	Squad IA
audit-service	8088	Auditoria & Compliance	PostgreSQL	Squad Governance

3.2 `auth-service` — Autenticação e Autorização

- **Repositório Git:** `git@github.com:santopegasus/auth-service.git`
- **Squad Owner:** Squad Hermes | **Tech Lead:** Isabella Carvalho
- **Responsabilidade:** Raiz de confiança do ecossistema. Emissão, validação e revogação de tokens JWT. Gestão do ciclo de vida de credenciais e fluxos OAuth 2.0 / OIDC. Toda autenticação passa obrigatoriamente por este serviço. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 8\]
- **Tecnologias:** Spring Boot 3.x + Security 6, JWT (RS256), PostgreSQL 15, Redis (Token Cache), Secrets Manager.
- **APIs Expostas:** Login, Refresh, Logout (Blacklist no Redis), OAuth2 Token, Validate (usado pelo Gateway), Password Reset. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 8\]

3.3 `user-service` — Gestão de Perfis de Usuários

- **Responsabilidade:** Gerencia perfis de pacientes, médicos, administradores e operadores. Fonte da verdade para dados de perfil (nome, contato, especialidades). Não armazena credenciais (exclusivo do `auth-service`). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 9\]
- **Tecnologias:** Spring Boot 3.x, PostgreSQL 15, Redis (Cache de Perfil), MapStruct, Swagger UI.
- **Modelo de Dados:** `users`, `doctors`, `patients`, `addresses`, `preferences`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 10\]

3.4 `agendio-scheduling-service` — Núcleo de Agendamento

- **Responsabilidade:** Serviço mais crítico. Gerencia lógica de criação, confirmação, cancelamento e reagendamento de consultas. Controle de agenda de médicos, conflitos de horários e regras de negócio de clínicas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 10\]
- **Tecnologias:** Spring Boot 3.x + JPA, PostgreSQL 15 (transações ACID), Flyway, Resilience4j.

- Dependências: `user-service` (validar atores), `payment-service` (verificar pagamento), `audit-service` (log de alteração), `agendiao-notification-service` (disparo de avisos). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 11\]

3.5 `agendiao-notification-service` — Notificações

- Responsabilidade: Serviço stateless que orquestra o envio de e-mails (AWS SES) e SMS (AWS SNS) baseados em preferências do usuário consultadas no `user-service`. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 12\]
- Tecnologias: Spring Boot 3.x + Spring Cloud AWS, Thymeleaf (Templates HTML), SQS Consumer.

3.6 `payment-service` — Processamento de Pagos

- Responsabilidade: Ciclo de vida de transações financeiras (Pix, Cartão de Crédito/Débito), integração com gateway externo (Stripe), estornos e recibos. Serviço idempotente e completamente auditado. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 13\]
- Tecnologias: Spring Boot 3.x, PostgreSQL 15, Stripe API, Banco Central API (Pix), Assinatura HMAC para Webhooks.

3.7 `medical-records-service` — Histórico Clínico (LGPD Compliance)

- Responsabilidade: Serviço mais sensível (proteção de dados). Gere anamneses, diagnósticos, prescrições e exames. Acesso controlado por RBAC e auditado. Consentimento explícito do paciente é obrigatório. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 14\]
- Tecnologias: Spring Boot 3.x + Security (RBAC granular), MongoDB (AWS DocumentDB), AWS S3 (anexos DICOM/PDF com URLs pré-assinadas), Criptografia em repouso (AWS KMS), Pipeline de anonimização (Art. 18 LGPD). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 14-15\]

3.8 `ai-assistant-service` — Assistente de IA com Arquitetura RAG

- Responsabilidade: Assistente de triagem inteligente para ajudar pacientes a identificar especialidades médicas com base em sintomas. Usa arquitetura RAG (Retrieval-Augmented Generation). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 16\]
- Tecnologias: LangChain4j, OpenAI GPT-4o, Pinecone (Vector Store para conhecimento médico), Redis (Cache de respostas).

3.9 `audit-service` — Registro de Auditoria e Compliance

- **Responsabilidade:** Registro histórico imutável de ações sensíveis. Consumidor puro (recebe dados via SQS). Registros arquivados no S3 Glacier após 90 dias. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 17\]
- **Tecnologias:** PostgreSQL 15 (immutable append-only), AWS S3 Glacier, SQS Consumer exclusivo.

SEÇÃO 4 — MAPA DE DEPENDÊNCIAS ENTRE SERVIÇOS

4.1 Tabela de Dependências \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 18\]

Serviço Consumidor	Serviço Provedor	Tipo	Protocolo	Criticidade
api-gateway	auth-service	Síncrono	REST / Redis Cache	Alta
agendio-scheduling-service	user-service	Síncrono	REST (WebClient)	Alta
agendio-scheduling-service	payment-service	Síncrono	REST (WebClient)	Alta
payment-service	audit-service	Assíncrono	SQS	Média
agendio-scheduling-service	agendio-notification-service	Assíncrono	SQS	Média
user-service	audit-service	Assíncrono	SQS	Média
medical-records-service	audit-service	Assíncrono	SQS	Alta

SEÇÃO 5 — PADRÕES DE COMUNICAÇÃO

5.1 REST Síncrono com WebClient

A Santo Pegasus utiliza Spring WebClient (não-bloqueante/reactivo) para comunicações síncronas. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 20\]

- **Regras:** mTLS obrigatório, Timeout de conexão (3s), Timeout de leitura (10s), Circuit Breaker (Resilience4j), Retry com backoff exponencial.

5.2 gRPC para Alta Performance

Adotado para comunicações de alta frequência e baixa latência entre serviços internos (ex: validação rápida de médico no `user-service`). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 20-21\]

5.3 Mensageria Assíncrona com AWS SQS

Padrão preferencial para eventos entre domínios. Utiliza SQS FIFO Queues para garantir ordenação e entrega exatamente-uma-vez (exactly-once delivery). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 21\]

- Padrões: Filas FIFO, DLQ configurada (maxReceiveCount = 3), Idempotência no consumidor, Envelope de evento padronizado.

SEÇÃO 6 — ESTRATÉGIA DE BANCOS DE DADOS

6.1 Database per Service Pattern

Cada microserviço controla exclusivamente sua própria base de dados. Integração entre domínios ocorre apenas via APIs ou mensageria. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 22\]

6.2 Mapa de Bancos de Dados por Serviço \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 22-23\]

- PostgreSQL 15 (RDS Multi-AZ): `auth\_db`, `users\_db`, `scheduling\_db`, `payments\_db`, `audit\_db`.
- MongoDB 6 (DocumentDB): `medical\_records\_db` (esquema flexível).
- Pinecone Cloud: `knowledge\_vectors` (Vector store para RAG).
- Redis 7 (ElastiCache): Cache Global (múltiplos namespaces).

SEÇÃO 7 — API GATEWAY

O API Gateway (Kong) é o único ponto de entrada para clientes externos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 23\]

- Responsabilidades: Roteamento, Autenticação Centralizada (JWT), Rate Limiting (100 req/min por tenant), SSL/TLS Termination (TLS 1.3), Request Logging com Trace ID, Transformações (Headers contextuais). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 23-24\]

SEÇÃO 8 — INFRAESTRUTURA NA AWS

8.1 Serviços AWS Utilizados \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 25\]

- ECS Fargate: Orquestração de contêineres Docker.
- RDS PostgreSQL Multi-AZ: Bancos relacionais com failover.
- S3 & S3 Glacier: Arquivos de exames e backups de auditoria (retensão 90+ dias).
- Secrets Manager & KMS: Gestão de credenciais e encriptação de dados sensíveis.
- VPC + Private Subnets: Isolamento de rede para microserviços (nunca expostos publicamente).

SEÇÃO 9 — OBSERVABILIDADE DISTRIBUÍDA

Baseada em três pilares: logs, métricas e rastreamento distribuído. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 26\]

- Trace ID: Propagado em headers HTTP ('X-B3-TraceId') e envelopes SQS para rastrear o ciclo de vida completo de uma requisição. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]
- Logging Estruturado: Formato JSON com campos padronizados. Proibido registrar PII (dados pessoais/sensíveis). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]
- Métricas: Spring Boot Actuator + Micrometer. Prometheus coleta via scraping e Datadog recebe via integração nativa. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 27\]

SEÇÃO 10 — ESTRATÉGIA DE VERSIONAMENTO DE APIS

- Versionamento Explícito: Via prefixo no path ('/v1/', '/v2/'). \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 28\]
- Processo de Depreciação: 3 fases (Anúncio -> Período de Suporte \[mês 1-6\] -> Sunset \[mês 6+ com HTTP 410 Gone\]). Versões obsoletas permanecem ativas por no mínimo 6 meses. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 29\]

SEÇÃO 11 — SEGURANÇA ENTRE SERVIÇOS

- **mTLS: Comunicação serviço-a-serviço dentro da VPC usa mTLS (Mutual TLS) com certificados rotacionados a cada 90 dias via AWS ACM Private CA. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 30\]**
- **IAM Roles: Cada microserviço opera com um IAM Role exclusivo seguindo o princípio do privilégio mínimo. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 30\]**
- **Scan de Segredos: Ferramentas como `gitleaks` e `trufflehog` bloqueiam commits com segredos expostos. \[Source: Arquitectura de Microservicios y Mapa de Dominios — Santo Pegasus Soluciones.pdf, Page 31\]**